Decoupled Information-Set MCTS with a Card-Value Model for Sushi Go!

Halit Ozger
Student no. [to be completed]
Queen Mary University of London
London, UK

[Group member 2]
Student no. [to be completed]
Queen Mary University of London
London, UK

[Group member 3]
Student no. [to be completed]
Queen Mary University of London
London, UK

***Abstract*—Card-drafting games such as *Sushi Go!* combine hidden hands, simultaneous moves and delayed scoring, which basic Monte Carlo tree search (MCTS) ignores. We extended BasicMCTS of the Tabletop Games framework (TAG) with card-value rollouts, a lead-based evaluation, round-end rollouts, max-n backups and a decoupled information-set tree. On held-out deals it wins 52.6% and 47.2% of three-player games (100, 500 ms) against TAG's information-set MCTS (ISMCTS) and our previous agent, and 44.4% and 36.5% of four-player games (100, 500 ms; chance 33.3%, 25%). Every ablated component raises the score margin significantly; the evaluation and the decoupled tree also raise the win rate. Longer budgets helped ISMCTS more than ours; four-player settings hurt three-player play (exploratory).**

Keywords*—*Monte Carlo tree search, information sets, simultaneous moves, card games, Sushi Go!, TAG

I. Introduction

Tabletop games combine chance, hidden information and several interacting players within small rule sets; the Tabletop Games framework (TAG) [1] gives many of them a common interface so that agents can be compared. Monte Carlo tree search (MCTS) [2] is the usual starting point for such agents, but its basic form assumes that the state is fully observed and that players move one after another [3]. *Sushi Go!*, one of TAG's games, breaks both assumptions: every player secretly picks one card from its hand, all players reveal their cards together and pass the rest of their hands on, and after three rounds the highest score wins. Klang et al. [4] implemented the game in TAG and described it as combining simultaneous action selection with complete information; in the TAG version we use, an agent's copy re-deals all unseen cards.

Four properties make the game hard for a search agent. (i) The opponents' hands are hidden until they are passed on. (ii) Moves are simultaneous: no card can be chosen in reaction to an opponent's card of the same turn. (iii) Many points are delayed and conditional – sets pay out only when completed, Maki rolls at the end of the round and Pudding at the end of the game – so short simulations miss them and long ones are noisy. (iv) Hands of nine (three players) or eight cards (four players) give up to 729 or 4096 joint moves per turn (more with Chopsticks), with 0.1–1 s per decision. TAG's BasicMCTS addresses none of the first three: it searches the one shuffled sample of the hidden cards it receives as if it were the true state, treats the players of a turn as moving one after another and evaluates random rollouts by the player's own score.

We ask which adaptations of MCTS to these properties make an agent stronger (RQ1), and how their value changes with the time per decision (RQ2) and the number of players (RQ3). We extend BasicMCTS in steps, keeping every modification as a switch, and test each in experiments of 500 games per condition (250 in some long runs). Our contributions are:

- a model of *Sushi Go!*'s scoring that estimates every card's marginal value, used as the rollout policy of all players, and every player's final score, used in a bounded, lead-based evaluation;
- its combination with round-end rollouts, max-n backups, decoupled UCT and information sets, compared with TAG's own decoupled information-set MCTS and tested by leave-one-out and factorial ablations and on held-out deals;
- exploratory evidence that settings tuned for four players hurt with three, and that with four players time beyond 500 ms rarely changes the decision of a wide, shallow joint-move tree.

II. Background

A. TAG and Sushi Go!

TAG [1] splits a game into a state, a forward model and parameters; an agent receives a copy of the state, in which every card it has not seen is reshuffled, and uses the forward model to simulate moves. In TAG's *Sushi Go!* the 108-card deck holds Tempura (5 points per pair), Sashimi (10 per triple), Dumplings (1, 3, 6, 10 or 15 points for one to five), Salmon, Squid and Egg Nigiri (2, 3 and 1 points, tripled on an earlier Wasabi), Maki rolls with one to three icons (6 and 3 points for the most and second-most icons of a round), Chopsticks (two cards in a later turn) and Pudding (+6 for the most and, with three or more players, −6 for the fewest at the end of the game, when Puddings also break ties). As the hands rotate, every hand of a round has been seen after two passes (three with four players), so hidden information is concentrated at the start of each round, while the value of a card depends on what the others will take from the hands we pass on.

B. Monte Carlo Tree Search

MCTS [2] builds a search tree by repeating selection, expansion, simulation (rollout) and backpropagation. Upper Confidence bounds applied to Trees (UCT) [5] selects the child that maximises the upper confidence bound (UCB1) $Q + K\sqrt{(\ln N / n)}$, where Q is the child's mean reward, n its visits and N those of its parent. Domain knowledge can enter through heuristic rollout policies or priors such as progressive bias [2], but a stronger rollout policy does not guarantee better estimates: Silver and Tesauro [6] showed that rollouts should be balanced, i.e., give unbiased value estimates.

C. Hidden Information, Simultaneous Moves and Several Players

Determinisation, or perfect-information Monte Carlo (PIMC) sampling, samples the hidden information, searches each sample as if it were the true state and combines the results. Long et al. [7] related its success in card games to properties such as how quickly hidden information is

All sections, tables and figures of this paper were drafted with Claude [15]; the prompts are given in the statement on the use of generative AI at the end of the paper.

revealed during play, which in *Sushi Go!* happens within two or three turns of each round. Its known flaws are strategy fusion – the search assumes that it can act differently in states it cannot tell apart – and non-locality [3], [7]. Information-set MCTS (ISMCTS) [3] avoids strategy fusion by searching one tree over the searcher's information sets and sampling the hidden information anew in every iteration. In its single-observer form the opponents' choices in the tree are not conditioned on their own private information; the multiple-observer form models it with one tree per player at a higher cost [3], an idea TAG's advanced MCTS agent offers as MultiTree MCTS [8].

Simultaneous moves can be modelled as hidden sequential moves (partially observable moves in ISMCTS [3]) or handled by decoupled UCT (DUCT): every player chooses its own action at a node from its own bandit statistics and the joint action leads to a child [9]. UCB-based DUCT need not converge to an equilibrium, so Lanctot et al. [9] compared it in Goofspiel with EXP3, regret matching and their online outcome sampling: regret matching and outcome sampling played best, and decoupled UCT's strategies were the most exploitable. Tak et al. [10] found across nine games that the best variant depends on the game and the opponent, although decoupled UCT was best overall. With several players, UCT with max-n backups (every player maximises its own value) computes a mixed-strategy equilibrium and played at least as well as earlier algorithms [11].

MCTS has also been applied to the drafting game 7 Wonders, which passes hands in the same way; many results known from classic games held there too, but card values depend on the stage of the game and on the other players' choices [12], which our card-value model estimates explicitly. We build on the minimal BasicMCTS, which lacks these features, so that each one's contribution can be measured. For *Sushi Go!* and similar drafting games we know of no study that separates the contributions of decoupled selection, information sets and domain knowledge within one agent or measures how they change with the time budget and the number of players; the earlier *Sushi Go!* study in TAG [4] focused on simultaneous action selection and complete information and concluded that better heuristics should improve the agents, which motivates our card-value model.

III. Method

Our agent, SushiMCTS, is a modified copy of TAG's BasicMCTS (package *players.basicMCTS*) and uses no code of TAG's advanced MCTS agent. Every modification is a switch in its parameter class (Table I). We compare the final agent (v4) with two earlier versions, v2 (H, A, B, D) and v3 (H, B, D, P, R, M), whose tree treats the players of a turn as moving in sequence (Fig. 1). The search techniques come from the literature; our own element is the card-value model shared by the evaluation and every player's simulation.

A. Card-Value Model and Heuristic (H)

For a state s and player p the model estimates the final score

$$\hat{S}_p = b_p + \alpha u_p + m_p + \pi\omega_t d_p, \quad (1)$$

where b_p is the banked score (TAG scores Tempura, Sashimi, Dumplings and Nigiri when they are revealed); u_p the value of unfinished sets, of unused Wasabi (2v, the bonus for a Nigiri of average value v = 2) and of Chopsticks (1), each weighted by $\min(1, (r - k + 1)/T)$, or 0 if $r < k$, when k more cards are needed and r picks are left; m_p the Maki points p would get if the round ended now; and d_p the Pudding (dessert) points if the game ended now, weighted by $\omega_t = t/3$ in round t ($\alpha = 0.5$, $T = 4$, $\pi = 1$). The value of s for player i is

$$H_i(s) = \tanh(\Delta_i / \sigma), \quad (2)$$

$$\Delta_i = w(\hat{S}_i - \max_{j \neq i} \hat{S}_j) + (1 - w)(\hat{S}_i - \mathrm{mean}_{j \neq i} \hat{S}_j), \quad (3)$$

with $\sigma = 15$ and $w = 0.7$: the first term rewards beating the leader, since only first place wins, and the second keeps a gradient when the agent is far ahead or behind. The marginal value $mv_p(c)$ of a card c is the change of $\hat{S}_p$ when p plays it.

TABLE I. Modifications of BasicMCTS (✓ = used in the final agent with 3 / 4 players)

Id	Modification	3 p / 4 p
H	heuristic (2)–(3) from model (1)	✓ / ✓
B	ε-greedy card-value rollouts for every player	✓ / ✓
R	rollouts end when the round is scored	✓ / ✓
P	one branch per distinct card	✓ / ✓
M	max-n backups [11]	✓ / ✓
S	decoupled UCT, one node per turn [9]	✓ / ✓
I	single-observer information-set search [3]	✓ / ✓
X0	four-player card constants [a]	– / ✓
W2	win-gap value for 4 players (Section III-D)	– / ✓ [b]
F, L	fast simulator, 8 rollouts per leaf	– / ✓ [b]
A	progressive bias [2]	dropped in v3
D	ensemble of up to 4 determinised trees [3]	replaced by I

[a] (σ, w, α, T, β, T_m, κ, δ, π, ν, γ) = (15, 0.7, 0.5, 6, 0.5, 3, 0.25, 1.5, 1.25, 3, 3), default (15, 0.7, 0.5, 4, 1, 0, 0, 0, 1, 2, 0). β: Sashimi triple needing two cards; Maki standing weight $1/(1 + r/T_m)$ (1 if $T_m = 0$), κ per icon otherwise; δ: next Dumpling; γ: cards needed per missing card. [b] 500 ms or more.

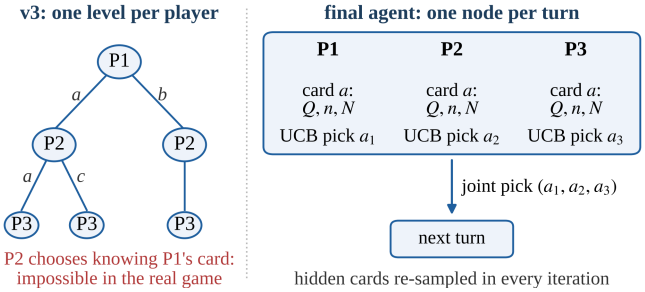


Fig. 1. The sequential tree of v3 (left) and a turn node of the final agent (right), where every player picks by UCB on its own card statistics (S, P) and the hidden cards are re-sampled in every iteration (I).

B. Simulation (B, R)

B: every player plays its card with the highest *mv*, or a random card with probability ε = 0.2 (the second card of a Chopsticks move in the same way, and greedily in the tree); the same policy models the opponents. **R:** a rollout stops when the current round has been scored; only Pudding carries over, which (1) captures, so at the leaves $u_p = m_p = 0$ and only banked points and the Pudding standing count.

C. Tree (P, M, S, I)

P: identical cards lead to identical states, so a node has one branch per distinct card. **M:** a node stores one value per player and every player maximises its own value [11], instead of BasicMCTS's paranoid assumption that every opponent minimises ours. **S:** one node represents a whole turn; at a node every player p picks a card by

$$a_p = \mathrm{argmax}_a \left[Q_p(a) + K\sqrt{(\ln N_p(a) / n_p(a))}\right] \quad (4)$$

from its own statistics, trying untried cards first; the joint choice is played in the forward model and leads to the child of that joint action [9]; in v3's tree a later player "saw" the card an earlier player had just chosen (Fig. 1). **I:** every iteration starts from a new sample of the cards we cannot see and walks one shared open-loop tree, whose nodes store

statistics but no states [3]; each iteration adds one node, and the agent finally plays the card it chose most often at the root. Statistics are keyed by card kind (type, Maki icons, use of Chopsticks); $n_p(a)$ counts how often p chose a at the node and the availability count $N_p(a)$ how many iterations reached the node with a in p's sampled hand (without I, the node's visits). I replaces the determinisation ensemble D of v2 and v3, which split the budget between up to four independent trees. With three players $K = 0.5$.

D. Four Players

With four players the agent switches parameters. It uses the card constants X0 (Table I), found by a search-free tuner of greedy card-value policies. From 500 ms it also uses **W2** – $w = 1$; finished games valued by $\tanh((b_i - \max_{j \neq i} b_j)/2)$ with Pudding as tie-break; σ multiplied by $(\rho/2)^{0.5}$ for ρ rounds left; opponents in the backup valuing their lead over the others' mean – and **F**, TAG's rules re-implemented on integer arrays and checked move by move against the forward model, whose speed is spent on **L** = 8 rollouts per leaf, with $K = 1$.

E. Experimental Setup

TABLE II. EXPERIMENTAL SETUP

Item	Setting
Framework	TAG as provided for the module (unchanged), *Sushi Go!* with its default rules; OpenJDK 21
Opponents	v2: our previous agent (H, A, B, D, K 1.41). TAG ISMCTS: SushiGo_3+P_HiBudget at our budget (one tree per player [8], UCB-Tuned [2], K 1, random rollouts to the end, score heuristic). TAG DUCT: SushiGoDUCT (decoupled UCT, K 1, information sets, lead heuristic, 30-move rollouts). BasicMCTS (K 1.41, 10-move rollouts), Rolling Horizon Evolutionary Algorithm (RHEA; horizon 10, population 10), One-Step Look-Ahead (OSLA, score heuristic)
Budget	CPU time per decision, equal for all agents: 100 ms (most development), 500 and 900 ms; submitted agent files: 900 ms (1 s limit) or 430 ms (500 ms limit), with wall-clock guards
Seating	our agent in seat g mod 3 in game g (4 players: g mod 4); the two opponents swap seats every 3 games (4 players: 2 + 1 copies, swapped every 4 games)
Deals	game g uses seed $s + g$ (seed block s); in all seed-7 and held-out runs a fixed deck order is shuffled with it (equal s, equal deals: paired runs); earlier runs used TAG's own deal (independent samples)
Seeds	development s = 1, 2 (v3 also 1001, 2001, 3001); ablation and later candidate tests s = 7; held-out s = 100 000, disjoint from all other deals
Games	500 per condition; 250 at 900 ms (development and seed block 7) and in four-player development at 500 ms (45–70 s of CPU time per game)
Hardware, records	18-core laptop (6–16 games in parallel); candidate tests and validation: 4-core machine (4 in parallel); per-game results and logs of all 165 runs: folder results of our code package

Each game is an independent trial whose outcome for our agent is 1 for a win, $1/j$ for a j-way shared first place and 0 otherwise. Win rates are reported with Wilson 95% confidence intervals (CI) [13]; the score margin (our score minus the best opponent's) as mean ± 1.96 standard errors (SE); differences between runs as the mean of the per-game differences on the same deals ± 1.96 SE (for independent samples, the difference of the means with the two-sample SE) and $z = |\text{difference}|/\text{SE}$, in percentage points (pp) for win rates. With 500 games a win rate near 50% has a 95% interval of ±4.4 pp, and a difference between two runs about ±5.7 pp even when paired, because the outcomes of two versions of our agent on the same deal correlate only weakly (at most 0.22 in our paired runs); such a comparison detects a difference of about 8 pp with 80% power. The score margin, usually the more sensitive measure, served to screen ideas. We report unadjusted intervals, apply a Holm correction [14] within each group of Fig. 2, and treat the four-player comparisons (60 of our 165 runs) as exploratory. We expected (E1) every component to improve both measures, (E2) the decoupled information-set tree to matter more at longer budgets and (E3) the best settings to depend on the number of players; E2 arose from the development runs.

IV. RESULTS

A. Final Agent (RQ1)

TABLE III. FINAL AGENT ON HELD-OUT DEALS AND AGAINST OTHER AGENTS

Setting	Opponents	Win % [95% CI]	Margin
3p/100	v2, ISMCTS	52.6 [48.2, 56.9]	+0.73 ± 0.46
3p/500	v2, ISMCTS	47.2 [42.9, 51.6]	−0.10 ± 0.40
4p/100	v2, ISMCTS	44.4 [40.1, 48.8]	−0.61 ± 0.43
4p/500	v2, ISMCTS	36.5 [32.4, 40.8]	−1.83 ± 0.45
3p/100	DUCT, ISMCTS	55.0 [50.6, 59.3]	+0.72 ± 0.40
3p/100	ISMCTS × 2 [a]	51.3 [46.9, 55.7]	+1.41 ± 0.51
3p/100	BasicMCTS, RHEA [a]	84.2 [80.7, 87.1]	+6.68 ± 0.60
3p/100	OSLA, BasicMCTS [a]	87.2 [84.0, 89.8]	+9.74 ± 0.75

Setting: players / ms per decision; 500 games each. Deals: held-out seed block 100 000; [a] seed block 7. Margin: our score minus the best opponent's (points, ± 95% CI); chance 33.3% (3p), 25% (4p).

Table III reports the final agent; in every setting its win rate is significantly above chance. On seed block 7 it had won 55.0% at 100 ms (58.1% when re-run on the second machine), within chance of the held-out result ($z \leq 1.8$). At 100 ms it scores more than v2 and ISMCTS from Nigiri (17.6 points per game against 15.3 and 12.9, Wasabi included) and Pudding, while v2 scores more from Sashimi (9.1 against 3.0). TAG's own DUCT agent (Table II) wins 21.0% [17.7, 24.8] of its seats in place of v2, the same share as v2 (0.0 ± 5.0 pp): TAG's decoupled information-set search, with its own heuristic and rollouts, did no better than our previous agent. BasicMCTS, RHEA and OSLA win at most 12.8% of their seats (OSLA none: its one-step look-ahead never reaches the reveal).

B. Ablation (RQ1)

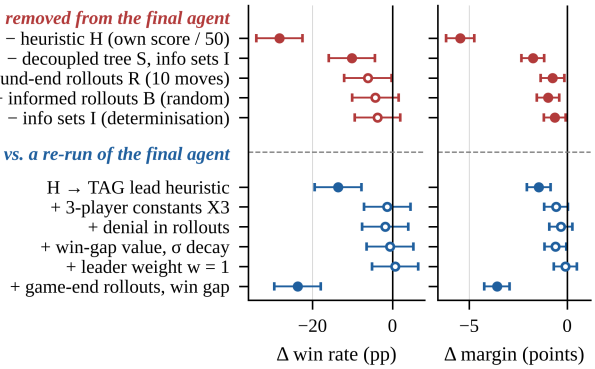


Fig. 2. Paired effects (three players, 100 ms, seed block 7, 500 games each; ± 95% CI; filled: significant after a Holm correction within the group). Top: components removed from the final agent (55.0%). Bottom: TAG's lead heuristic and five later changes against its re-run (58.1%).

Fig. 2 (top) removes one component at a time from the final agent. Every removal lowers both measures; on the score margin all five differences survive a Holm correction, on the win rate only those of H and of S + I, so for these five components E1 holds for the margin and only partly for wins. Without H the leaves are valued by TAG's default heuristic

(own score / 50); as K was not retuned to its scale, the 28.3 ± 5.8 pp loss mixes the value of H with mis-scaled exploration. TAG's lead heuristic (own score minus the best opponent's, K rescaled to 7.5) recovers 17.8 ± 5.6 pp of this loss (same deals, second machine) but still wins 13.6 ± 5.9 pp (z = 4.5) less than H; at round-end leaves, which hold no unfinished sets, H differs from it mainly by the Pudding standing, the mean-opponent term and its bounded scale. S and I together are worth 10.2 ± 5.7 pp (z = 3.5) here and +6.1 ± 5.9 pp on the held-out deals (pooled +8.2 ± 4.1 pp, z = 3.9). S alone, decoupled selection with v3's determinisation, beats v3 by +6.4 ± 5.7 pp (z = 2.2, unadjusted), and I adds a further, non-significant 3.8 ± 5.7 pp, consistent with Long et al.'s analysis [7] for a game that reveals its hidden information within two turns of each round. P and M stay; in a 2^4 factorial on v3 (R, M, K = 0.5 or 1.41, T = 8 or 4; 16 × 500 games) R added 8.7 ± 2.1 pp and M 4.1 ± 2.0 pp, with R × T and M × K interactions (z = 4.0, 2.8).

Five later changes (Fig. 2, bottom) brought no detectable gain: card constants tuned for three players (X3), denial of the next player's favourite card in the rollouts, a win-gap value with σ decay and w = 1 changed the win rate by −1.8 to +0.7 pp (each ±5.8), although X3 is the much stronger greedy policy (45.0% of games against two X0 players; it also changes the leaf evaluation, so this only weakly echoes simulation balancing [6]); rollouts to the game end with a win-gap value cost 23.7 ± 5.8 pp (z = 8.0).

C. Time Budget (RQ2)

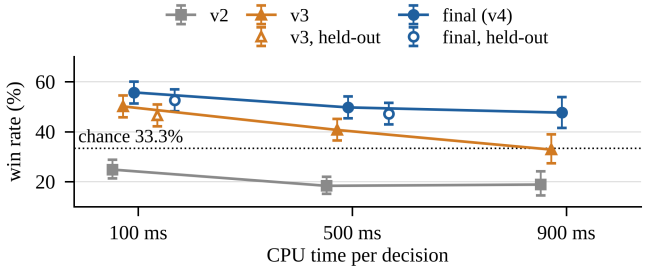


Fig. 3. Three-player win rate against v2 and TAG's ISMCTS. Lines: development runs (seed 1, independent samples; 250 games at 900 ms); open markers: the final agent and v3 on the held-out deals; bars: Wilson 95% intervals.

In development (Fig. 3; independent samples) the gain from S and I rose from +5.6 ± 6.1 pp at 100 ms to +14.8 ± 8.4 pp at 900 ms (by +9.2 ± 10.4 pp, z = 1.7), as v3 lost 17.3 pp and the final agent 8.1 pp. We hypothesise that v3's decline comes from its sequential tree, in which later players react to cards chosen earlier in the same turn: a wrong model that more iterations exploit more. As v3's 50.1% at 100 ms was the best of 16 factorial cells (45.1–49.4% on new seeds), this rise is probably overstated. On the held-out deals S and I gained +6.1 ± 5.9 pp (z = 2.0) at 100 ms (v3: 46.5%).

Giving every agent more time did not help ours: on the held-out deals our win rate changed by −5.4 ± 5.9 pp (z = 1.8) at 500 ms against 100 ms; pooled with seed block 7, 500 ms changed it by −5.5 ± 4.2 pp (z = 2.6). On the held-out deals ISMCTS won 26.4% and 35.9% of its seats at 100 and 500 ms (500 against 100 ms: +9.5 ± 5.7 pp), v2 21.0% and 16.9%: the extra time favoured ISMCTS over our agent. Plausibly, more iterations cannot correct the bias of our hand-made rollouts and evaluation, while ISMCTS, without such a model, keeps gaining from more samples; our parameters were also tuned at 100 ms.

D. Four Players and the Limits of Search (RQ3)

No four-player setting gave a significant gain in development: X0 changed the win rate by +4.3 ± 4.3 pp (z = 1.9) at 100 ms, X0 with W2 and K = 1 by +5.8 ± 7.4 pp at 500 ms, and F + L a further +1.0 ± 7.9 pp at 500 ms but −6.6 ± 6.0 pp at 100 ms; we kept each change where its estimate was positive. With three players at 500 ms (both arms with F + L), X0, W2 and K = 1 cost 7.7 ± 5.4 pp (z = 2.8), an effect 13.5 ± 9.1 pp (z = 2.9) different from that with four. This is consistent with E3 but exploratory: the test was chosen after the four-player runs, whose gains were not significant, and only the three-player arms had F + L.

With four players we found no gain from more time. Depth matters – at 500 ms a tree cut to depth 1 lost 17.4 ± 8.2 pp – but 1000 ms for our agent alone (with K = 1.37) brought no detectable gain (−4.8 ± 8.1 pp; 80% power only for 12 pp), and the 500 ms agent (W2, F + L, K = 1; all agents at 500 ms) won 6.7 ± 4.1 pp less than the 100 ms agent (z = 3.2; seed block 7 and held-out deals pooled), while TAG's ISMCTS rose from 24.1% to 32.1% of its seats, as with three players. Both agents did better on seed block 7 (46.8% and 41.4%), whose deals overlap those used to tune them. The tree statistics suggest why: on 73 positions from random-play games the 500 ms tree has on average 704 joint-move children at the root, 53% of them visited once, and doubling the time or the rollouts per leaf changed the chosen card in 2 and 4 of the positions, no more often than repeated runs did (4 and 7).

V. Conclusions and Future Work

For RQ1, every ablation lowers the score margin; after a Holm correction, removing the evaluation H or the decoupled information-set tree also lowers the win rate (by 28.3 and 10.2 pp; pooled with the held-out replication the tree is worth 8.2 ± 4.1 pp). For RQ2, giving every agent more time did not help our agent: its win rate changed by −5.5 ± 4.2 pp from 100 to 500 ms (two deal sets pooled), while TAG's ISMCTS won 9.5 pp more of its seats at 500 ms; in development v3's sequential tree lost more strength at longer budgets than the decoupled one, though the difference between the trends is not significant. For RQ3, settings tuned for four players cost 7.7 ± 5.4 pp with three players (exploratory); and with four players time beyond 500 ms rarely changes the decision of the wide, shallow joint-move tree.

Our opponents were framework agents and our own previous version, the four-player 500 ms settings were kept on non-significant 250-game gains, and CPU-time budgets hide wall-clock effects. As hand-tuned changes to the model no longer move the agent, future work should learn the evaluation from game outcomes; regret matching or EXP3 deserve a test, as decoupled UCT is best overall but not in every game [10] and was the most exploitable in Goofspiel [9]; and opponent models fitted to observed picks (an option in our code, not yet evaluated in full runs) could replace the single greedy policy that now models all players.

Use of Generative AI

We used Claude [15] (Anthropic), through the Claude Code tool, throughout the project: to write and modify the Java code of our agent on top of BasicMCTS, to write and run the experiment and statistics scripts, and to draft all sections, tables and figures of this paper from the logs of our runs; every number comes from the result files of our runs. The authors take full responsibility for the code and the paper, including any errors. The paper was generated with

the prompt (original Turkish, then English translation): *“simdi reponun icindeki butun herseyi incele ve hocaya direk yuklyecegimiz raporuda olusutr repoda ornekler tipler brief hersey var 100 alcagimiz bir rapor olsun her raporu tamaladiginda baska bir agent spawn edip bagimsiz durusstce kontrol taki yuzde yuz alacagana emin olankadar looptan cikma”* – “Now examine everything in the repository and also create the report we will upload directly to the lecturer; the repository has examples, tips, the brief, everything. It should be a report that gets 100. Every time you finish the report, spawn another agent to check it independently and impartially, and do not leave the loop until you are sure it will get 100%.” Following that prompt, each draft was marked by a second, independent Claude instance, prompted as “an independent, strict marker for the QMUL module ECS7032P” to mark it against the brief and the rubric, to recompute its numbers from our result files and to list the changes that would raise the mark. Its written reports, one per draft, were the prompts for the revisions (the prompts of this report and these reports: GENAI_PROMPTS.md and the folder genai in our code package); they asked, among other things, for validation on deals disjoint from development, a correct description of the availability-count UCB, all replicate runs, intervals and hedges for non-significant trends, the comparison with TAG's DUCT agent, the lead-heuristic ablation, v3 on the held-out deals, the four-player diagnostics, correct figure references, this statement's citation and prompts, claims no stronger than the statistics and the selection bias of v3's development run. The later candidate changes were tested with the prompt “The lecturer will hold a tournament of our agents in class; as before, improve the agents' performance quickly while following the lecturer's restrictions and recording what we did in the README; you have 4.5 hours, test everything in parallel.” The prompts of earlier development sessions are not reproduced here; they asked Claude to implement and test the modifications of Table I and to run the experiments of Section IV.

References

[1] R. D. Gaina, M. Balla, A. Dockhorn, R. Montoliu, and D. Perez-Liebana, “TAG: A tabletop games framework,” in *Proc. AIIDE Workshop on Experimental AI in Games (EXAG)*, 2020.

[2] C. B. Browne *et al.*, “A survey of Monte Carlo tree search methods,” *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, 2012.

[3] P. I. Cowling, E. J. Powley, and D. Whitehouse, “Information set Monte Carlo tree search,” *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 2, pp. 120–143, 2012.

[4] C.-M. E. Klang, V. Enhörning, A. Alvarez, and J. Font, “Assessing simultaneous action selection and complete information in TAG with Sushi Go!,” in *Proc. IEEE Conf. Games (CoG)*, 2021, pp. 1–4.

[5] L. Kocsis and C. Szepesvári, “Bandit based Monte-Carlo planning,” in *Proc. 17th Eur. Conf. Mach. Learn. (ECML)*, 2006, pp. 282–293.

[6] D. Silver and G. Tesauro, “Monte-Carlo simulation balancing,” in *Proc. 26th Int. Conf. Mach. Learn. (ICML)*, 2009, pp. 945–952.

[7] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, “Understanding the success of perfect information Monte Carlo sampling in game tree search,” in *Proc. 24th AAAI Conf. Artif. Intell.*, 2010, pp. 134–140.

[8] J. Goodman, D. Perez-Liebana, and S. Lucas, “MultiTree MCTS in tabletop games,” in *Proc. IEEE Conf. Games (CoG)*, 2022, pp. 292–299.

[9] M. Lanctot, V. Lisý, and M. H. M. Winands, “Monte Carlo tree search in simultaneous move games with applications to Goofspiel,” in *Computer Games (CGW 2013)*, Commun. Comput. Inf. Sci., vol. 408. Cham, Switzerland: Springer, 2014, pp. 28–43.

[10] M. J. W. Tak, M. Lanctot, and M. H. M. Winands, “Monte Carlo tree search variants for simultaneous move games,” in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, 2014, pp. 232–239.

[11] N. R. Sturtevant, “An analysis of UCT in multi-player games,” in *Computers and Games (CG 2008)*, Lect. Notes Comput. Sci., vol. 5131. Berlin, Germany: Springer, 2008, pp. 37–49.

[12] D. Robilliard, C. Fonlupt, and F. Teytaud, “Monte-Carlo tree search for the game of ‘7 Wonders’,” in *Computer Games (CGW 2014)*, Commun. Comput. Inf. Sci., vol. 504. Cham, Switzerland: Springer, 2014, pp. 64–77.

[13] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[14] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[15] Anthropic, “Claude” [large language model, used through the Claude Code tool], Oct. 2026. [Online]. Available: https://claude.ai (accessed Oct. 28, 2026).